

Assignment 2 – Theory

Rotem Beker 217386598

Bar-Ilan CS 89-3312, Parallel Systems Programming

1 Question 1: Balanced Parentheses

We wish to determine whether the parentheses in a given mathematical expression are balanced. We present a parallel algorithm that decides this in time $O(\log n)$, using n threads, where n is the length of the input string.

1.1 Algorithm Description

The key insight is to encode each character as an integer and reduce the problem to a prefix-sum check.

1. Map every character of the input string to an integer:
 - $(\mapsto +1$
 - $) \mapsto -1$
 - any other character $\mapsto 0$
2. Build an integer array A of length n , where $A[i]$ holds the value assigned to character i . Apply a parallel *prefix-sum* on A ; call the result S .
3. $S[i]$ now equals the running total of open-minus-close parentheses up to position i .
4. The parentheses are balanced if and only if:
 - $S[n-1] = 0$ (equal number of $($ and $)$).
 - $\forall 0 \leq i \leq n-1 : S[i] \geq 0$ (no prefix has more closing than opening brackets).

1.2 Pseudocode

```
1: procedure BALANCEDPARENTHESES(string  $str$ , integer  $n$ )
2:   parallel for  $i = 0$  to  $n - 1$ :  $A[i] \leftarrow \text{encode}(str[i])$ 
3:    $S \leftarrow \text{ParallelPrefixSum}(A)$ 
4:   if  $S[n-1] \neq 0$  then return false
5:   end if
6:   parallel for  $i = 0$  to  $n - 1$ :
7:     if  $S[i] < 0$  then return false
8:   end if return true
9: end procedure
```

1.3 Proof of Correctness

Claim: The expression has balanced parentheses $\Leftrightarrow$ the algorithm returns *true*.

($\Leftarrow$) Balanced $\Rightarrow$ algorithm returns true. Let the input be a balanced expression. By definition, every opening parenthesis has a matching closing one, so the total count is equal: $S[n-1] = 0$, and the first condition holds.

Suppose for contradiction that $S[i] < 0$ for some index i . Then the prefix up to i contains strictly more $)$ than $($, which means a closing parenthesis appears without a matching opener — contradicting the assumption that the expression is balanced. Hence $S[i] \geq 0$ for all i , and the algorithm returns *true*.

($\Rightarrow$) Algorithm returns true $\Rightarrow$ balanced. Assume the algorithm returns *true*; we show no violation is possible.

1. If some $)$ has no matching $($, then some prefix has more closing than opening brackets, so $S[i] < 0$ for that prefix — the algorithm would have returned *false*. Contradiction.
2. If some $($ has no matching $)$, then $S[n-1] > 0 \neq 0$ — the algorithm would have returned *false*. Contradiction.

Therefore the expression is balanced.

1.4 Runtime Analysis

- Step 1 (build A): n threads work in parallel — $O(1)$.
- Step 2 (prefix-sum): $O(\log n)$.
- Step 3 (check $S[n-1]$): single comparison — $O(1)$.
- Step 4 (check all $S[i] \geq 0$): n threads in parallel — $O(1)$.

Total: $O(\log n)$.

2 Question 2: Counting Subarrays with a Given Sum

Given an array A of length n and a target b , count the number of contiguous subarrays whose sum equals exactly b .

The core mathematical identity is:

$$\text{sum}(i, j) = S[j] - S[i - 1] = b \iff S[j] = S[i - 1] + b,$$

where S is the prefix-sum array of A (with $S[-1] = 0$). Hence, for each index j we need to count indices $i \leq j$ with $S[i - 1] = S[j] - b$.

2.1 Part 1: Positive Numbers Only

2.1.1 Algorithm Description

When all elements are strictly positive, the prefix-sum array S is *strictly increasing*. This allows binary search without any prior sorting step.

1. Compute S via a parallel prefix-sum on A .
2. For each index j (in parallel), set $\text{target}_j = S[j] - b$.
3. Since S is sorted, use binary search to find the range $[\ell, r]$ where $S[\cdot] = \text{target}_j$.
4. The number of valid starting indices for subarrays ending at j is $r - \ell + 1$ (or 0 if no such range exists). Store this in $C[j]$.
5. Return $\sum_j C[j]$ via a parallel prefix-sum on C .

2.1.2 Pseudocode

```

1: procedure COUNTSUBARRAYS_POSITIVE(array  $A$ , integer  $n$ , integer  $b$ )
2:    $S \leftarrow \text{ParallelPrefixSum}(A)$   $\triangleright S$  is strictly increasing
3:   parallel for  $j = 0$  to  $n - 1$ :
4:      $t \leftarrow S[j] - b$ 
5:      $\ell \leftarrow \text{LowerBound}(S[0..j - 1], t)$ 
6:      $r \leftarrow \text{UpperBound}(S[0..j - 1], t)$ 
7:      $C[j] \leftarrow \max(0, r - \ell + 1)$ 
8:   return  $\text{ParallelPrefixSum}(C)[n - 1]$ 
9: end procedure

```

2.1.3 Proof of Correctness

Claim: Every contiguous subarray $A[i..j]$ with sum b is counted exactly once.

Every valid subarray is counted ($\Leftarrow$). Let $A[i..j]$ have sum b . Then $S[j] - S[i - 1] = b$, so $S[i - 1] = S[j] - b = t_j$. Since S is strictly increasing, $S[i - 1]$ is the unique occurrence of t_j in $S[0..j - 1]$, so the binary search finds it and $C[j]$ is incremented.

Every counted pair is valid ($\Rightarrow$). Suppose the binary search finds index $i - 1 \in [0, j - 1]$ with $S[i - 1] = t_j = S[j] - b$. Then $\text{sum}(i, j) = S[j] - S[i - 1] = b$ and $i \leq j$, so $A[i..j]$ is a valid subarray.

2.1.4 Runtime Analysis

- Parallel prefix-sum: $O(\log n)$.
- Binary search for each j in parallel: $O(\log n)$.
- Final summation via prefix-sum: $O(\log n)$.

Total: $O(\log n)$.

2.2 Part 2: General Case (Arbitrary Integers)

2.2.1 Algorithm Description

When elements may be negative or zero, S is no longer monotone and must be sorted before binary search. We use a stable parallel sort to preserve original indices.

1. Check (in parallel) whether all $A[i] \geq 0$; set a flag *needSort* accordingly.
2. Compute S via a parallel prefix-sum on A . Each entry stores its original index: $(S[i], i)$.
3. If *needSort* is true, apply a stable parallel sort on S by value.
4. For each index j (in parallel), compute $\text{red}_j = S[j].\text{val} - b$.
5. For each j (in parallel), binary-search for the range $[first, last]$ in the sorted S where the value equals red_j .
6. Within that range, binary-search the original indices to count entries whose original index is strictly less than j ; call this count $C[j]$.
7. Return $\text{ParallelPrefixSum}(C)[n - 1]$.

2.2.2 Pseudocode

```

1: procedure COUNTSUBARRAYS_GENERAL(array  $A$ , integer  $n$ , integer  $b$ )
2:    $needSort \leftarrow$  parallel check  $\exists i : A[i] < 0$ 
3:    $S \leftarrow \text{ParallelPrefixSum}(A)$  ▷ store pairs  $(S[i], \text{origIdx} = i)$ 
4:   if  $needSort$  then  $S \leftarrow \text{StableParallelSort}(S)$ 
5:   end if
6:   parallel for  $j = 0$  to  $n - 1$ :
7:      $red \leftarrow S[j].\text{val} - b$ 
8:      $first \leftarrow \text{LowerBound}(S, red)$ 
9:      $last \leftarrow \text{UpperBound}(S, red)$ 
10:     $size \leftarrow$  last index in  $S[first..last]$  s.t.  $\text{origIdx} < j$  ▷ binary search on origIdx
11:     $C[j] \leftarrow size - first + 1$ 
12:   return  $\text{ParallelPrefixSum}(C)[n - 1]$ 
13: end procedure

```

2.2.3 Why Stable Sort?

Within a block of equal values in the sorted S , entries are ordered by their original index (thanks to stability). This makes the inner binary search on original indices correct, since the block is sorted by index.

2.2.4 Proof of Correctness

Every valid subarray is counted ($\Leftarrow$). Suppose $\text{sum}(i, j) = b$, i.e. $S[j] - S[i - 1] = b$ with $i - 1 < j$. Thread j sets $red = S[j] - b = S[i - 1]$. After sorting, $S[i - 1]$ lies in the range $[first, last]$. Since $i - 1 < j$, it satisfies the original-index filter and is counted in $C[j]$.

Every counted pair is valid ($\Rightarrow$). Thread j counts an entry e only if $e.\text{val} = red = S[j] - b$ and $e.\text{origIdx} < j$. Setting $i - 1 = e.\text{origIdx}$ gives $\text{sum}(i, j) = S[j] - e.\text{val} = b$, a valid subarray.

2.2.5 Runtime Analysis

- Step 1 (positivity check): $O(1)$ in parallel.
- Step 2 (prefix-sum): $O(\log n)$.
- Step 3 (stable parallel sort, if needed): $O(\log^2 n)$.
- Steps 4–6 (binary searches in parallel): $O(\log n)$.
- Step 7 (final sum): $O(\log n)$.

Total: $O(\log n)$ (positive-only case), $O(\log^2 n)$ (general case).

3 Question 3: k -th Ancestor in a Rooted Tree

Given a rooted tree of n nodes represented by a parent-pointer array (the root points to itself) and an integer k , find for every node its k -th ancestor. Return -1 if no such ancestor exists.

Goal: parallel runtime $O(\log k)$ using n threads.

3.1 Algorithm Description

A naïve step-by-step climb takes $O(k)$. We exploit the binary representation of k : since $k = \sum_{j: b_j=1} 2^j$, we can accumulate jumps of size 2^j for each set bit.

We maintain two arrays:

- $loc[i]$: current position of node i after processing the bits seen so far.
- $prog[i]$: the ancestor of node i at distance 2^j (doubles every iteration).

3.2 Pseudocode

```

1: procedure KTHANCESTOR(parent[0..n-1],  $n$ ,  $k$ )
2:   parallel for  $i = 0$  to  $n - 1$ :
3:      $loc[i] \leftarrow i$ 
4:     if  $parent[i] = i$  then  $prog[i] \leftarrow -1$ 
5:     else  $prog[i] \leftarrow parent[i]$ 
6:     for  $j = 0$  to  $\lfloor \log_2 k \rfloor$  do
7:       if  $(k \gg j) \ \& \ 1 = 1$  then ▷ bit  $j$  of  $k$  is set
8:         parallel for  $i = 0$  to  $n - 1$ :
9:           if  $loc[i] \neq -1$  and  $prog[loc[i]] \neq -1$ :
10:             $loc[i] \leftarrow prog[loc[i]]$ 
11:          else:  $loc[i] \leftarrow -1$ 
12:         end if
13:         parallel for  $i = 0$  to  $n - 1$ : ▷ double the jump size
14:            $cur \leftarrow prog[i]$ 
15:           if  $cur \neq -1$  and  $prog[cur] \neq -1$ :
16:              $prog[i] \leftarrow prog[cur]$ 
17:           else:  $prog[i] \leftarrow -1$ 
18:         end for
19:   return  $loc$ 
20: end procedure

```

3.3 Proof of Correctness

Claim: A k -th ancestor of v exists $\Leftrightarrow$ the algorithm returns it in $loc[v]$; otherwise $loc[v] = -1$.

($\Rightarrow$) **Algorithm returns non-(-1) $\Rightarrow$ correct ancestor.** Each iteration j in which bit j is set applies a jump of exactly 2^j (by the doubling invariant on $prog$). The total distance traversed is

$$\sum_{j: b_j=1} 2^j = k.$$

Since the algorithm returned a non-(-1) value, all jumps stayed within tree bounds. Therefore $loc[v]$ is the ancestor of v at distance exactly k .

($\Leftarrow$) **k -th ancestor exists $\Rightarrow$ algorithm finds it.** If $k \leq \text{depth}(v)$, every partial jump stays within the tree, so $prog[loc[v]] \neq -1$ at every relevant step, and the updates accumulate to distance k .

Out-of-bounds case. If $k > \text{depth}(v)$, some jump would exceed the root. At that step $\text{prog}[\text{loc}[v]] = -1$, the algorithm sets $\text{loc}[v] \leftarrow -1$, and this value propagates to the end.

3.4 Runtime Analysis

- Initialisation (step 1): $O(1)$ in parallel.
- Main loop: $\lfloor \log_2 k \rfloor + 1$ iterations, each $O(1)$ in parallel — total $O(\log k)$.

Total: $O(\log k)$.

4 Question 4: Trapping Rain Water

Given an array $height[0..n-1]$ of non-negative integers representing bar heights of width 1, compute the total units of water trapped after rain.

4.1 Algorithm Description

The water above position i equals

$$w[i] = \max(0, \min(lmax[i], rmax[i]) - height[i]),$$

where $lmax[i] = \max_{j \leq i} height[j]$ and $rmax[i] = \max_{j \geq i} height[j]$.

We compute $lmax$ via a left-to-right parallel prefix-max, and $rmax$ via a right-to-left parallel prefix-max (equivalently, reverse the array, prefix-max, reverse back). Both are standard $O(\log n)$ operations.

4.2 Pseudocode

```

1: procedure TRAPPINGRAINWATER( $height[0..n-1]$ ,  $n$ )
2:    $lmax \leftarrow \text{ParallelPrefixMax}(height)$  ▷ left-to-right:  $lmax[i] = \max_{j \leq i} height[j]$ 
3:    $rmax \leftarrow \text{ParallelPrefixMax}(\text{reverse}(height))$ , then reverse  $rmax$  ▷  $rmax[i] = \max_{j \geq i} height[j]$ 
4:   parallel for  $i = 0$  to  $n - 1$ :
5:      $w[i] \leftarrow \max(0, \min(lmax[i], rmax[i]) - height[i])$ 
6:   return  $\text{ParallelPrefixSum}(w)[n - 1]$ 
7: end procedure

```

4.3 Proof of Correctness

Claim: $\sum_i w[i]$ equals the total trapped water.

Lower bound (no overcounting). $w[i] \geq 0$ by the $\max(0, \cdot)$ guard, so no negative water is added. $w[i] \leq \min(lmax[i], rmax[i]) - height[i]$: the water level at i cannot exceed the shorter of the two surrounding walls, and the bar itself occupies $height[i]$ units.

Upper bound (no undercounting). For any position i where $\min(lmax[i], rmax[i]) > height[i]$, there exist bars at or to the left and at or to the right of i that are at least as tall as $\min(lmax[i], rmax[i])$. These bars form a container that retains exactly $\min(lmax[i], rmax[i]) - height[i]$ units of water above position i .

Together, $w[i]$ is the exact water contribution of position i , and summing over all positions gives the total trapped water.

Example verification. For $height = [0, 2, 0, 3, 1, 0, 1, 3, 2, 1]$:

i	0	1	2	3	4	5	6	7	8	9
height	0	2	0	3	1	0	1	3	2	1
lmax	0	2	2	3	3	3	3	3	3	3
rmax	3	3	3	3	3	3	3	3	2	1
$w[i]$	0	0	2	0	2	3	2	0	0	0

$$\sum w[i] = 0 + 0 + 2 + 0 + 2 + 3 + 2 + 0 + 0 + 0 = 9 \checkmark$$

4.4 Runtime Analysis

- Parallel prefix-max for $lmax$: $O(\log n)$.
- Reverse + parallel prefix-max + reverse for $rmax$: $O(\log n)$.
- Computing $w[i]$ in parallel: $O(1)$.
- Parallel prefix-sum for total: $O(\log n)$.

Total: $O(\log n)$.